



Навчально-науковий інститут атомної та
теплової енергетики

Кафедра інженерії програмного забезпечення в
енергетиці

Основи Веб-програмування. Курсова робота
Веб-додаток "PassGuard": Менеджер безпечних паролів

Виконав: Білоус Андрій Олегович, група ТВ-43

Перевірів: Недашківський О. Л.

GitHub репозиторій [посилання](#)

2026



Актуальність теми

Курсова робота присвячена розробці безпечного веб-додатку — менеджера паролів.

Основна задача – створення платформи для генерації надійних паролів, їх перевірки на стійкість до зламу та безпечного зберігання в особистому зашифрованому сховищі користувача.

Значення теми для галузі інженерії програмного забезпечення:

Розробка менеджерів паролів є критично важливою для галузі інженерії програмного забезпечення в умовах постійного зростання кіберзагроз. Вона демонструє інтеграцію сучасних методів криптографії у зручний веб-інтерфейс, вирішує проблему використання слабких паролів та підкреслює першочергове значення захисту конфіденційних даних користувачів.

Мета роботи: Розробити функціональний та захищений веб-додаток для генерації та управління паролями.

Предмет роботи: Процеси розробки веб-додатку, вибір технологій, створення адаптивного інтерфейсу та реалізація механізмів авторизації, ізоляції даних та криптографічного захисту.

Об'єкт роботи: Веб-додаток "PassGuard" на основі фреймворку Django, з використанням SQLite/PostgreSQL для зберігання даних та Bootstrap 5 для користувацького інтерфейсу.



Постановка задачі

Перелік задач, функцій, які потрібно було розв'язати для досягнення мети курсової роботи:

Реєстрація та авторизація користувачів: Реалізація можливості створення облікових записів із безпечним хешуванням паролів. Забезпечення перевірки унікальності логінів. Захист доступу до внутрішніх сторінок додатка.

Генератор надійних паролів: Розробка гнучкого інструменту для миттєвого створення випадкових паролів із налаштуванням їхньої довжини (від 8 до 32 символів) та вибором набору символів (цифри, літери, спецсимволи).

Управління сховищем: Надання можливості зберігати згенеровані паролі у прив'язці до конкретних сервісів. Забезпечення ізоляції даних — кожен користувач бачить виключно свої записи.

Перевірка надійності: Розробка інтерактивного аналізатора для оцінки складності та стійкості вже існуючих паролів користувача в режимі реального часу.

Управління профілем: Надання користувачам можливості редагувати свої облікові дані (логін, пошту) та змінювати майстер-пароль.

Адміністрування: Створення закритої панелі керування для управління базою користувачів та їхніми даними.



Аналіз предметної області

Огляд існуючих рішень та їх недоліків

Вбудовані менеджери браузерів

- Зручні, але прив'язують користувача до однієї екосистеми та мають слабкий захист при фізичному доступі до пристрою.

Хмарні комерційні сервіси

- Потужні, але є платними сервісами із закритим вихідним кодом. Ризик масштабного витоку даних у разі зламу централізованих серверів компанії.

Десктопні офлайн-програми

- Висока безпека, але відсутність зручного доступу з різних пристроїв без ручної синхронізації файлів бази даних.

Обґрунтування вибору технологій та інструментів

Python / Django: Потужний веб-фреймворк, який ідеально підходить для додатків, де безпека на першому місці. Має вбудовані інструменти для захисту від CSRF, XSS та SQL-ін'єкцій, а також готову надійну систему автентифікації.

SQLite / PostgreSQL: Надійна реляційна база даних, що чудово працює з Django ORM, забезпечуючи ефективне та структуроване управління збереженими даними користувачів.

Bootstrap 5 / JavaScript: JS забезпечує миттєву генерацію паролів на стороні клієнта, а Bootstrap гарантує створення сучасного, інтуїтивно зрозумілого та адаптивного інтерфейсу для різних пристроїв.



Проектування системи

Способи/методи/методики вирішення поставленої задачі

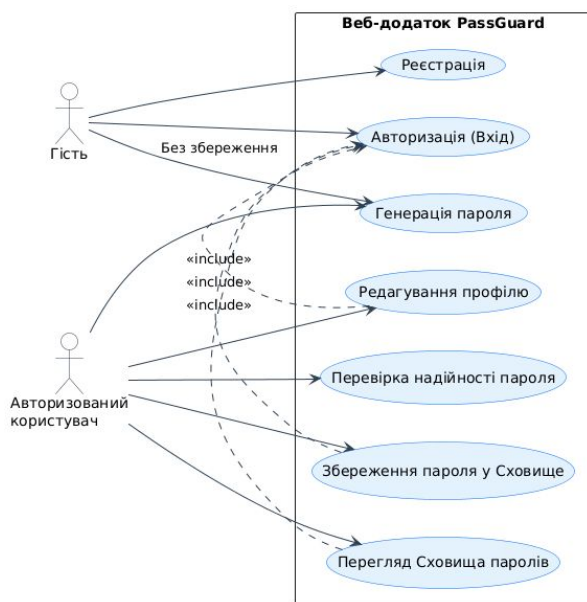
Для розробки веб-додатку менеджера паролів були використані наступні методики та підходи:

- Архітектурний підхід MVT: Розподіл системи на Моделі (робота з БД), Представлення (бізнес-логіка) та Шаблони (інтерфейс), що забезпечує чистоту коду.
- Криптографічний захист: Використання стандарту хешування PBKDF2 для збереження майстер-паролів, що робить неможливим злам методом перебору.
- Генерація на клієнті: Перенесення логіки генератора паролів на сторону клієнта (JS), щоб паролі створювалися миттєво і не передавалися мережею до моменту натискання кнопки "Зберегти".

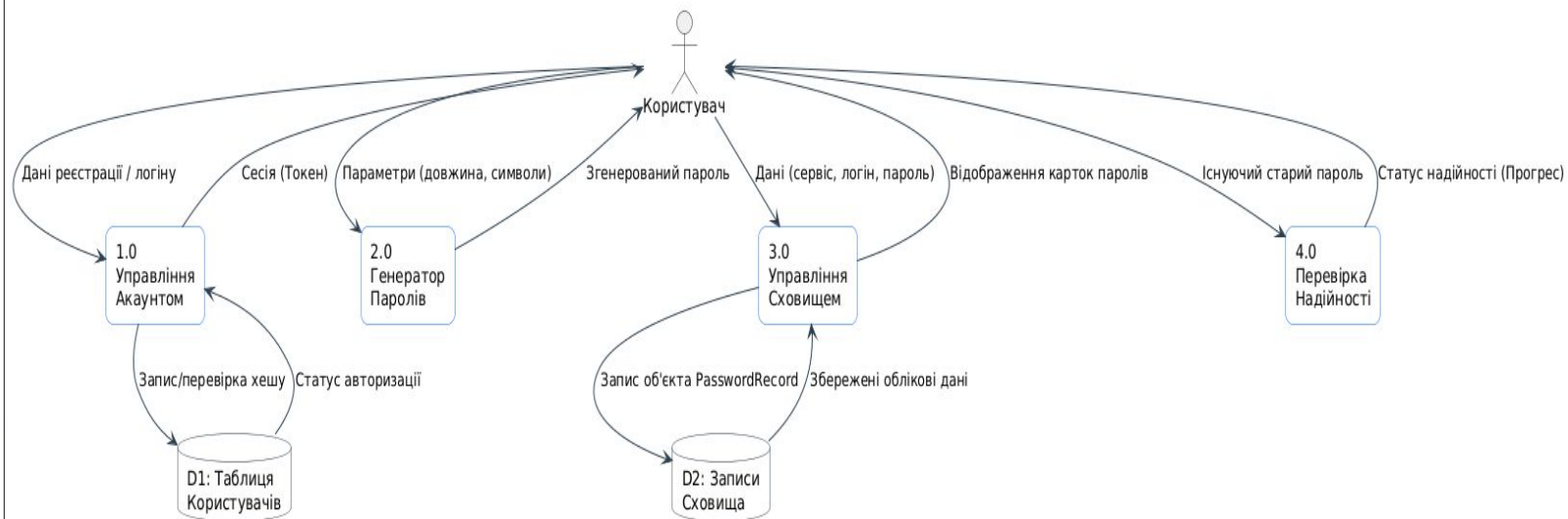


Проектування системи

Діаграми



Діаграма
Use-case



Діаграма DFD



Проектування системи

Методи, які було використано в роботі:

Спосіб зберігання паролів: Для того, щоб ніхто не міг викрасти паролі, програма використовує вбудовані методи захисту Django. Паролі не зберігаються у чистому вигляді, а перетворюються на спеціальний зашифрований код.

Робота з базою даних: Використано реляційну модель. Це означає, що кожен збережений пароль чітко прив'язаний до свого власника (користувача), і ніхто інший його не побачить.

Архітектура системи

Програма побудована за стандартною схемою Django (MVT):

- Model (База даних): Описує, як виглядають дані (таблиці користувачів та паролів).
- View (Логіка): Основний код, який обробляє натискання кнопок, перевіряє логін та керує генератором.
- Template (Інтерфейс): Те, що бачить користувач у браузері (сторінки сайту).



Проектування системи

Вимоги до системи (функціональні та нефункціональні) Функціональні вимоги:

- Реєструвати нових людей та впускати їх на сайт за логіном/паролем.
- Генерувати паролі (можна вибирати довжину та символи).
- Зберігати ці паролі в особистому списку.
- Показувати список усіх збережених паролів.
- Перевіряти «на око», наскільки старий пароль користувача надійний.
- Давати змогу змінити свою пошту чи пароль у профілі.
- Можливість адміністрування (зміна даних будь-якого користувача суперкористувачем).

Нефункціональні вимоги:

- Надійність: Щоб сайт не "падав" і дані не губилися.
- Швидкість: Паролі мають генеруватися миттєво.
- Зручність: Сайт має однаково добре виглядати і на комп'ютері, і на телефоні.



Реалізація системи

Опис процесу розробки

Розробка веб-додатку менеджера паролів включала кілька етапів:

Збір ідей:

- Вирішив, які кнопки та функції потрібні в менеджері паролів.

Малювання схем:

- Зробив діаграми, щоб зрозуміти, як користувач буде ходити по сайту.

Написання коду (Backend):

- Налаштував Django, створив базу даних та прописав логіку входу на сайт.

Створення вигляду (Frontend):

- Написав HTML-сторінки та додав JavaScript, щоб генератор паролів працював прямо на екрані без перезавантаження.

Тестування:

- Перевірів, чи все зберігається правильно і чи не можна зайти в чужий акаунт.



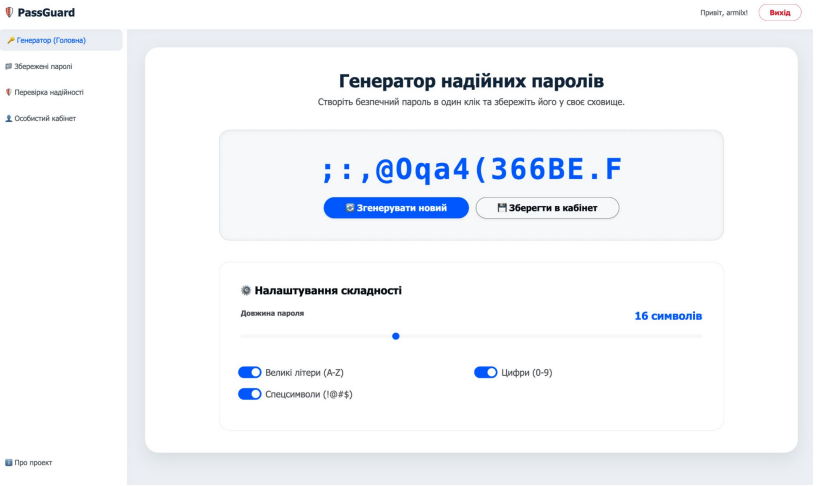
Реалізація системи

Мови та інструменти, які я використав:

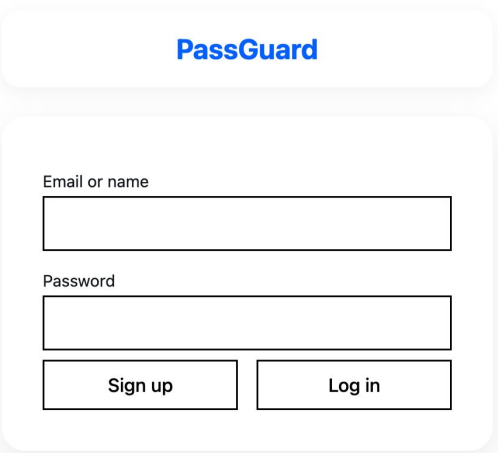
- Python: Основна мова, на якій написаний весь сайт.
- Django: Головний "двигун" (фреймворк), який відповідає за безпеку та роботу з базою.
- JavaScript: Потрібен для того, щоб паролі генерувалися миттєво і смужка надійності рухалася в реальному часі.
- HTML та CSS: Для створення структури та красивого дизайну сторінок.
- Bootstrap 5: Набір готових стилів, щоб сайт був сучасним і зручним.
- SQLite: Проста база даних для зберігання всієї інформації.
- Role-Based Access: Реалізація різних рівнів доступу для звичайних користувачів та адміністраторів.



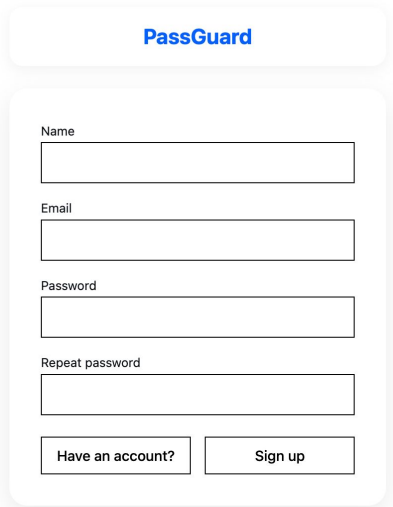
Реалізація системи



Головна сторінка сайту



Сторінка входу в додаток



Сторінка створення нового акаунта



Реалізація системи

Ваше сховище паролів

Service	Action	Login	Password
Netflix	Занес.	panda	*****
telegram	Занес.	bilous	*****
youtube	Занес.	ar@gmail.com	*****

Сховище паролів

Перевірка надійності пароля

Введіть свій поточний пароль, щоб дізнатися, наскільки він безпечний. Ми нікуди його не зберігаємо.

7:I4t=j*A,0M2J&5

Надійний пароль ✓

Перевірка надійності паролів



Реалізація системи

PassGuard Привіт, admin! Вихід

Генератор (Головна)
Збережені паролі
Перевірка надійності
Особистий кабінет
Панель Адміна

Панель Адміністратора Режим Адміна

Тут ви бачите всіх користувачів сайту та можете керувати їхніми даними.

Логін	Поточний Email	Змінити Email	Змінити Пароль
andrii	bilousandrii@gmail.com	Нова пошта Оновити	Новий пароль Змінити
armilx		Нова пошта Оновити	Новий пароль Змінити

Адміністративна панель

Особистий кабінет

Зміна особистих даних

Логін Електронна пошта

Оновити дані

Зміна майстер-пароля

Новий пароль Повторіть пароль

Введіть новий пароль Повторіть ще раз

Змінити пароль

Особистий кабінет



Тестування та верифікація

Методи тестування

Для забезпечення якості та надійності системи були використані наступні методи тестування:

1. Перевірка функціоналу:

-Тестування всіх основних кнопок. Генератор створює паролі точно за заданими правилами (довжина, наявність цифр/символів), а Сховище коректно записує та видає інформацію.

2. Перевірка надійності:

-Жорстке тестування доступу. Гість не може зайти в Сховище. Звичайний користувач бачить тільки свої паролі і не має доступу до чужих. Адмін-панель захищена і відкривається тільки для суперкористувача.

3. Перевірка вигляду:

-Сайт тестувався на різних пристроях. Завдяки Bootstrap 5, сторінки автоматично підлаштовуються під розмір екрана смартфона чи комп'ютера.

Результати тестування

- Усі функції працюють правильно. Дані надійно зберігаються, помилок під час взаємодії користувача із сайтом не виявлено. Сторонні користувачі не мають доступу до чужих паролів.

Оцінка якості та надійності системи:

- **Якість:** Інтерфейс інтуїтивно зрозумілий. Усі кнопки і форми працюють без затримок.

- **Надійність:** Використання фреймворку Django забезпечує високий захист від зломів, а дані користувачів надійно ізолювані.

Порівняння з аналогами:

Мій додаток простіший у використанні, не вимагає платної підписки та дає користувачу повний контроль над його даними, на відміну від багатьох хмарних сервісів.



Впровадження та супровід

Процес розгортання системи

Процес розгортання системи включає наступні кроки:

1. **Встановлення середовища:** Встановлення Python на комп'ютер.
2. **Клонування репозиторію:** Завантаження коду проекту з GitHub.
3. **Встановлення залежностей:** Встановлення фреймворку Django через командний рядок.
4. **Міграція бази даних:** Виконання команди `python manage.py migrate` для створення таблиць.
5. **Запуск додатку:** Запуск локального сервера командою `python manage.py runserver` та відкриття сайту в браузері.

Документація для користувачів та розробників

Документація для користувачів:

-Інструкція з реєстрації, використання генератора паролів та управління особистим сховищем у звіті курсової роботи.

Документація для розробників:

-Опис структури проекту (README на GitHub), інструкція із запуску та роботи з базою даних.

Плани щодо подальшого розвитку та підтримки системи

Подальший розвиток системи включає:

- Розширення функціоналу:** Додавання розширення для браузера (щоб паролі самі підставлялися на сайти).
- Покращення інтерфейсу користувача:** Додавання темної теми для зручності.
- Розробка мобільної версії:** Створення мобільного додатку для Android та iOS.
- Підтримка безпеки:** Впровадження входу за відбитком пальця або двофакторної автентифікації (2FA).



Висновки

Основні результати роботи

Розроблено веб-додаток менеджера паролів "PassGuard" з такими можливостями:

- Реєстрація та безпечна авторизація користувачів.
- Генерація складних паролів за заданими параметрами.
- Збереження паролів в особистому ізольованому сховищі.
- Перевірка надійності старих паролів.
- Адмін-панель для управління обліковими записами.

Досягнення мети та виконання завдань

Проект досяг мети, створивши зручний та безпечний інструмент для роботи з паролями. Всі завдання виконані: реалізовано реєстрацію, створено генератор, налаштовано адмін-панель та забезпечено безпечне зберігання даних у базі SQLite/PostgreSQL.

Практична та наукова цінність

Практична цінність – допомога користувачам у захисті їхніх акаунтів від зламу без необхідності запам'ятовувати десятки складних паролів.

Наукова цінність – застосування сучасних методів шифрування та безпеки у веб-розробці на базі Django.

Перспективи подальших розробок

Подальший розвиток проекту:

- Впровадження автоматичних сповіщень про те, що пароль застарів і його час змінити..
- Створення функції безпечного обміну паролем з іншими користувачами.
- Подальша оптимізація швидкості роботи та безпеки системи.